Blockchain

Concept of a Blockchain in Contrast to a Database - main purpose is study the structure of a blockchain

A blockchain is similar to a database in the fact that they both store information. However, a blockchain specializes in being decentralized and secure, with a focus on storing transactions. Transactions being a change in state, such as the transfer of money from one person to another's bank accounts. Adding information requires a lot more computation, so often you store large pieces of information in a database and have pointers to it in the blockchain.

Understand how a blockchain works internally (blocks, chains, transactions)

- What is a Blockchain "Block" and what are the data in a block. How/when is a new block formed?

First, we start with the concept of a distributed ledger, which is an append-only, publicly distributed digital record of the blockchain. Each "node" in the overall system has a copy of the ledger. The blockchain is basically a data structure, composed of units called "blocks". Each block contains metadata that allows verification of the information in the block's body, which primarily contains transactions.

New blocks go through two protective measures before being formed: verification and maintenance. When a new transaction is made, it is broadcasted to all nodes, which confirm its validity (no double spending, no negative balances, etc). If valid, it's added to the mempool, a list of pending transactions.

The next step is to put the transactions in a new block, and add the block. It's important to have a secure system to do this, otherwise anyone could just add blocks to create the most "updated" version of the ledger, and do malicious things like erasing a transaction they made.

There are two main systems: proof of work and proof of stake. Proof of work relies on the fact that a majority (51%) of computing power (miners) are good. The miners solve complex hashing problems, and the correct ledger is seen as the longest one. If a majority of miners are good, then they will always solve the math faster and be creating the longest ledger.

The second is proof of stake. When a block is added, people who sign up to be verifiers put in a stake, or a certain amount of their money. A verifier is then randomly chosen, with some weight on their amount of stake, who then proposes a block to be added. The other verifiers verify the correctness of the block, and if a supermajority (⅔) agree, the block is added.

Validators who vote incorrectly or act maliciously will have their stake slashed, which incentivizes them to behave correctly.

As for when a block is added, it depends majorly on the type of blockchain.

- How are Blocks "chained" together?

Blocks are chained together through hashing. Each block stores the hash of the previous block. When verifying a blockchain, you can go block by block using the hash to see if it is correct.

- What are transactions in terms of the data held within?

Again, transactions are a change in state. Inside they often include the sender, the recipient, what's being transferred, and a digital signature for authenticity.

Learn the difference between "permissioned" and "permissionless" blockchains.

As the names imply, permissionless blockchains are completely open, anyone can participate as a node or view the distributed ledger. Permissioned blockchains restricts who can view/add data to the blockchain.

Understand the concept of HASHING and how data hashing is the process used to create "chains" of "Blocks".

Hashing is essentially a function that deterministically takes an input and creates an output. There are a few requirements for a hash function:

- First preimage resistant: given h(x) = y, it should be computationally infeasible to find x
- Second preimage resistant: given h(x1) = y, it should be computationally infeasible to find x2 such that h(x2) = y
- Collision resistant: no two inputs should ever result in the same output

Fundamentally this concept allows for secure verification of objects, as you just have to share the hash of an object (which won't reveal what the object is). Any verifiers just have to hash their object and see if it matches.

Learn the concept of a Smart Contract. Programming languages used to create smart contracts

- State Variables vs. Transaction Data

A smart contract is a self-executing program stored on the blockchain. Often it creates state variables, such as a bank balance, which are permanent in memory. When transactions come in, they call a function such as withdraw() to change the state, and the transaction data is then erased. The most popular language for programming smart contracts is Solidity. The language depends on the blockchain and if it can be compiled into chain-readable bytecode.

Data Structures - Hash Tables, World State Database

Core Block Structure:

- Linked list: stores blocks
- Merkle tree: verification of transactions
- Array: stores transactions

State Layer:

- Account Trie (Merkle-Patricia Trie): maps account addresses to account data
- Merkle-Patricia Trie: per contract state variable storage

Consensus:

- Hash Table/Priority Que: mempool
- Set/List/Key-Value Storage: proof of stake tracking

Public and Private Key Cryptography

Public and private key cryptography is one of the main forms of encryption. The keys are inverses of each other, as the encryption using one key can be decrypted with the other. However as the name implies often the private key is only known to one party and the public key known to everyone. In terms of blockchain, they can be used as signatures, where a user encrypts/signs a transaction with their private key, and the nodes decrypt it with the public key to make sure it’s from the correct user. There are many mathematical methods of private/public key cryptography, a popular one being RSA, but those details are not necessary for now.

Hyperledger Fabric

Hyperledger fabric is a blockchain designed for enterprise use firsthandedly. This is represented through it being permissioned, modular, confidential, and high-throughput. The modularity allows for plug-and-play of ordering service, membership service, peer-to-peer gossip, etc, fitting many different needs. One of the most distinct architectural differences of Fabric is the use of the execute-order-validate consensus protocol instead of the usual order-execute, which has a few nodes run the transactions first, then have them ordered by consensus. This prevents the need for all nodes to execute transactions, as well as allows the use of standard programming languages since nondeterministic languages aren't a problem anymore.

For Hyperledger Fabric what are peers, orderers and clients?

Peers are the individual nodes that make up the blockchain network. Each peer has a copy of the ledger and chaincode (usually known as smart contracts). Orderers are nodes that order and package a list of executed and signed transactions into blocks, which are then sent to each peer. They are vital in the consensus process. Clients are the actual users of the blockchain, and access the blockchain through their peer node.

For Hyperledger Fabric what are "Channels"?

Channels are basically a grouping of peers that have access to a specific blockchain. Each channel has its own ledger and chaincode, and the peers that have access to that channel have a copy of each. Peers not in the channel cannot access that specific blockchain.

For Hyperledger Fabric what is a Channel Event?

I wasn't able to find channel events on the documentation, but there are chaincode events, which can allow applications to be notified of changes to the blockchain without polling.

For Hyperledger Fabric understand the function of the Membership Service Provider(MSP) Note: Advanced Topic

Pros:

- Built for enterprise
- Modular
- Channels provide granular control over data permissions
- Permissioned blockchain

- Identity management
- Can write smart contracts with common languages: (Node, Go, Java)

Cons:

- Higher overhead, more complex to set up
- Need external querying logic like Couchdb
- No native token system

Ethereum

Ethereum is a widely used public blockchain, relying heavily on monetary transactions. It uses proof of stake for consensus, and requires a payment called gas fees whenever you send a transaction. For enterprise purposes and data privacy, a private Ethereum blockchain should be used.

Pros:

- Larger dev ecosystem
- Native tokenization
- Easy to anchor to public blockchain (public Ethereum) for third parties/authorities to verify

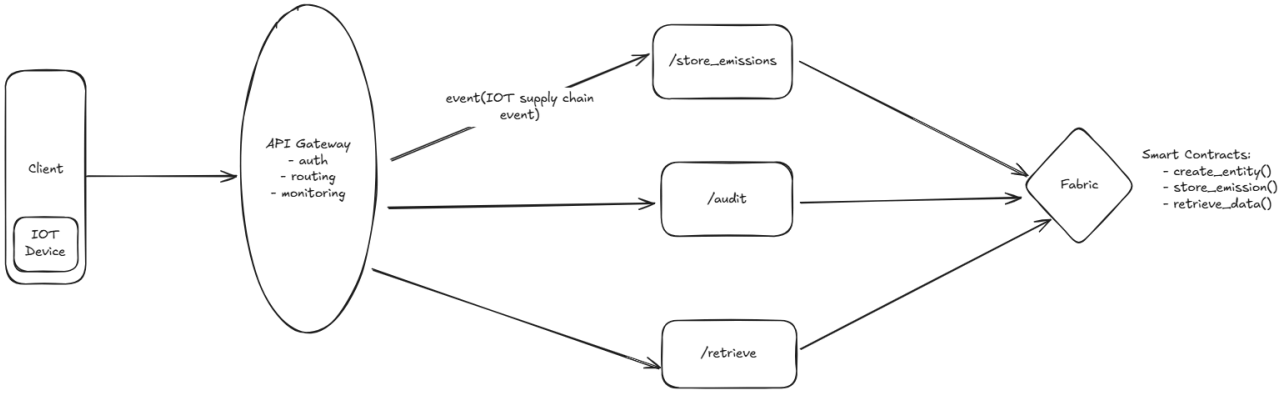
Cons:

- Need to rework Gas fees/Proof of Stake
- Solidity for smart contracts

Decision:

Due to the built-in channel use, being permissioned, enterprise-oriented, and modular, Fabric is the choice I would make for this project. The granular level control of who can see what information makes Fabric the primary choice in a business environment, where secrecy is crucial. The only main concern is the lack of native tokenization, in the case that some carbon credit system needs to be created. However it seems like the requirements only need integration with an external carbon credit platform, so this should not be too big of a concern.

Architectural Pattern:



The basic architectural pattern for this digital twin application will be a microservice run on servers. This will ensure scalability and modularity as a supply chain will contain multiple entities, each with multiple processes a day. We use servers instead of serverless as we assume this will be run at a large scale, so paying for uptime is the better option.

Each service will interface with Fabric using a few core smart contract functions. The functions should be as modular as possible so that all core functionalities can be achieved with a mix of them.

The microservices presented in this diagram are only an example, and the details have yet to be flushed out. However the basic concept is that some services will be event-driven, such as when a supplier's IOT device sends shipping information, which should be processed into carbon emission data and stored in Fabric. The audit (used for official third parties to verify carbon emissions) and retrieval (used for an analytics dashboard) services can be done on function call.

Other concerns:

- Does each separate supply chain need its own Fabric blockchain instance? How do we tell when a supply chain is its own instance, what if two chains end up intersecting?
- Synchronization concerns, reading before some emissions are stored